UNIVERSIDAD NACIONAL DE SAN AGUSTÍN DE AREQUIPA

FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS | EPIS
TÓPICOS AVANZADOS EN INGENIERÍA DE SOFTWARE

GUÍA OPERATIVA Y PROTOCOLO DE EJECUCIÓN EXPERIMENTAL

Evaluación de Rendimiento, Consumo de Recursos y Tolerancia a Fallos entre Monolito Modular y Microservicios: Caso Reserva de Tickets

Asignatura:	Tópicos Avanzados en Ingeniería de Software (TAIS)	**Docente:**	M.Sc. Ing. Robert Edison Arisaca Mamani
Integrantes:	• Marco Antonio Ponce de León Aguilar • Braulio Armando Chavez Nina • Emerson Danny Mendoza Hilasaca • Yimy Alfredo Charca Ccama	**Finalidad:**	Guía de laboratorio para captura de evidencias y recolección de métricas numéricas (Hito 2)

1. PRERREQUISITOS DEL ENTORNO DE PRUEBAS

Cualquiera de los integrantes del equipo puede ejecutar el experimento en su computadora verificando previamente:

- **Docker Desktop:** Instalado y en ejecución (asegurarse de que el motor Docker y Docker Compose v2 estén activos).
- **Grafana k6:** Descargado e instalado (añadido al PATH) desde https://k6.io/ o vía paquete winget install k6.
- **PowerShell:** Terminal predeterminada de Windows (ejecutar como Administrador si es necesario para políticas de scripts).
- **Ruta del Proyecto:** Ubicarse en la consola en la carpeta raíz del experimento: C:\Proyectos\TAIS\ticket-architecture-benchmark

Comandos de verificación en terminal:

```
docker --version
docker compose version
k6 version
```

2. MÉTODO 1: EJECUCIÓN TOTALMENTE AUTOMATIZADA (RECOMENDADO)

Hemos preparado un script orquestador maestro que realiza **todo el experimento de forma autónoma**: levanta el Monolito Modular, recolecta métricas de CPU y RAM, ejecuta las pruebas de carga con k6, apaga el entorno, espera un enfriamiento de 15 segundos, repite el mismo proceso para Microservicios y exporta los archivos CSV consolidados a la carpeta benchmark/results/.

Comando único para ejecutar todo el experimento:

```
powershell -ExecutionPolicy Bypass -File benchmark/scripts/run-all-benchmarks.ps1
```

3. MÉTODO 2: EJECUCIÓN MANUAL PASO A PASO (PARA CAPTURA DE PANTALLAS)

Si desean tomar capturas de pantalla individuales de cada fase para los Anexos del informe, sigan estos pasos:

3.1. Tratamiento A: Monolito Modular

Paso 1: Levantar entorno

```
docker compose -f docker-compose.monolith.yml up --build -d
```

Paso 2: Comprobar salud

```
curl http://localhost:3000/health
curl http://localhost:3000/api/events?limit=5
```

Paso 3: Iniciar muestreo de CPU/RAM (en terminal 2)

```
powershell -ExecutionPolicy Bypass -File benchmark/scripts/collect-metrics.ps1 -Environment "monolith"
-DurationSeconds 180
```

Paso 4: Ejecutar prueba de carga nominal (10 a 200 VUs)

```
k6 run benchmark/k6/load-test.js
```

Paso 5: Ejecutar prueba de sobreventa/concurrencia (150 VUs)

```
k6 run benchmark/k6/stress-test.js
```

Paso 6: Apagar y limpiar datos

```
docker compose -f docker-compose.monolith.yml down -v
```

3.2. Tratamiento B: Microservicios

Paso 1: Levantar los 3 servicios y PostgreSQL

```
docker compose -f docker-compose.microservices.yml up --build -d
```

Paso 2: Comprobar Gateway y eventos

```
curl http://localhost:3000/health
curl http://localhost:3000/api/events?limit=5
```

Paso 3: Iniciar muestreo de CPU/RAM (en terminal 2)

```
powershell -ExecutionPolicy Bypass -File benchmark/scripts/collect-metrics.ps1 -Environment "microservices"
-DurationSeconds 180
```

Paso 4: Ejecutar prueba de carga nominal (10 a 200 VUs)

```
k6 run benchmark/k6/load-test.js
```

Paso 5: Ejecutar prueba de sobreventa/concurrencia (150 VUs)

```
k6 run benchmark/k6/stress-test.js
```

Paso 6: Prueba de Tolerancia a Fallos e Inyección de Caídas

```
# En terminal 1: lanzar prueba de resiliencia
k6 run benchmark/k6/resilience-test.js

# En terminal 2 (a los 35s): detener el servicio de reservas
docker stop benchmark_ms_reservations
# (Observar que GET /api/events sigue respondiendo 200)

# Reiniciar y medir recuperación
docker start benchmark_ms_reservations
```

Paso 7: Apagar y limpiar datos

```
docker compose -f docker-compose.microservices.yml down -v
```

4. AUDITORÍA DE MANTENIBILIDAD CON SONARQUBE

Para obtener los índices de mantenibilidad, complejidad ciclomática y deuda técnica para la tabla comparativa del informe:

Comando para escanear el proyecto en SonarQube:

```
sonar-scanner -Dproject.settings=benchmark/sonar-project.properties
```

5. PLANTILLA PARA LLENAR LA SECCIÓN 'RESULTADOS' EN EL INFORME (HITO 2)

Una vez ejecutados los scripts, copien los valores promedio y percentiles en estas tablas para su documento Word:

Tabla 1: Comparativa de Rendimiento y Desempeño (k6)

Métrica	Monolito Modular	Microservicios	Diferencia Relativa (%)
Latencia Media (ms)	[Valor k6 avg]	[Valor k6 avg]	[Diferencia %]
Latencia Percentil 95 - p95 (ms)	[Valor k6 p95]	[Valor k6 p95]	[Diferencia %]
Latencia Percentil 99 - p99 (ms)	[Valor k6 p99]	[Valor k6 p99]	[Diferencia %]
Throughput Máximo (RPS / req/s)	[Valor k6 req/s]	[Valor k6 req/s]	[Diferencia %]
Tasa de Error bajo Carga (%)	[k6 error rate %]	[k6 error rate %]	[Comparación]
Consistencia / Cero Overbooking	0% sobreventa (20/20)	0% sobreventa (20/20)	Paridad idéntica

Tabla 2: Comparativa de Consumo de Recursos (Docker Stats CSV)

Métrica de Recurso	Monolito Modular	Microservicios	Diferencia / Impacto
Uso de CPU Medio (%)	[CSV CPUPerc avg]	[Suma CPUPerc avg]	[Diferencia %]
Uso de CPU Pico (%)	[CSV CPUPerc max]	[Suma CPUPerc max]	[Diferencia %]
Uso de Memoria RAM Basal (MB)	[CSV MemUsage min]	[Suma MemUsage min]	[Multiplicador RAM]
Uso de Memoria RAM Pico (MB)	[CSV MemUsage max]	[Suma MemUsage max]	[Multiplicador RAM]

Tabla 3: Tolerancia a Fallos y Aislamiento de Caídas

Escenario de Evaluación	Monolito Modular	Microservicios
Disponibilidad de Catálogo ante Caída de Reservas	0% (Falla en proceso detiene todo)	>99.5% (Aislamiento de fallos en Gateway)
Tiempo Medio de Recuperación - MTTR (s)	[Tiempo de reinicio de contenedor]	[Tiempo de reinicio de servicio reservas]